$x_1, \ldots, x_m$. So if locations 3 and 6 in the input are 1, we would expect the network to detect that $(x_3 \wedge x_6)$ is satistied.

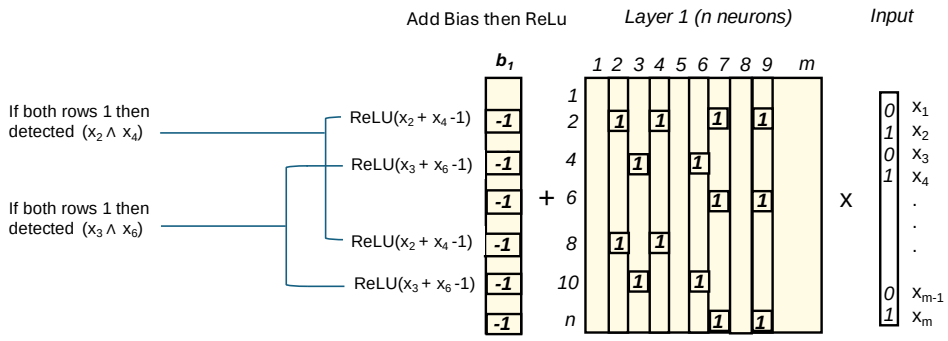

Figure 1: A neural network that computes $(x_3 \wedge x_6)$, $(x_2 \wedge x_4)$, and $(x_7 \wedge x_9)$ using Feature Channel Coding for each AND [1].

Consider how the feature $x_3 \wedge x_6$ is computed. The network does this by using a code for the two inputs x_3 and x_6 in columns 3 and 6 of the weight matrix. Specifically, the code places a 1 value in both of those columns for the two neurons 4 and 10. All other entries in the two columns are 0. The key to feature channel coding is that those two codes are the same (columns 3 and 6 are identical), and so the set of rows (neurons) where 1s appear in those columns can be used to compute the AND of those two inputs. Consider now what happens when the input consists of $x_3 = x_6 = 1$ (True), and all other variables are 0. When all n neurons of the layer compute the dot product with this input, neurons 4 and 10 will produce a sum of 2 each (before the bias) while all other neurons will produce 0. If we then add the bias of -1 and perform a ReLU for those neurons, we obtain a 1, as desired. More generally, for any combination of 0s and 1s for inputs x_3 and x_6, both neuron 4 and neuron 10 will produce a post ReLU value of $\text{ReLU}\,(x_3 + x_6 - 1) = x_3 \wedge x_6$. We can think of the code consisting of those two neurons as serving as a "computational channel" for computing $x_3 \wedge x_6$, and this can be detected by the next layer of the network (which is not depicted in the figure).

Similarly, we use a code to detect the output feature $(x_2 \wedge x_4)$ and that code does not overlap with the code for $(x_3 \wedge x_6)$. However, as the number of features grows, the codes will begin to overlap, as is the case of the codes for $(x_2 \wedge x_4)$ and $(x_7 \wedge x_9)$, which overlap in neuron 2. This introduces "noise" into the computational channel, and for that reason it can be helpful to use larger codes (i.e., codes with 1s in a larger number of neurons), so as to reduce the impact of this noise. [1] shows how to select random codes that will create feature channels that provably avoid having too much noise from overlaps so as to ensure there are no false positive detections and that the signal of the correct feature codes is not too diminished.

3.2 Neural Networks Learn Feature Channel Coding

One of the main open questions from [1] was whether these kinds of codes occur naturally in networks trained via gradient descent. In other words, does an SGD trained neural network learn to use feature channel coding?

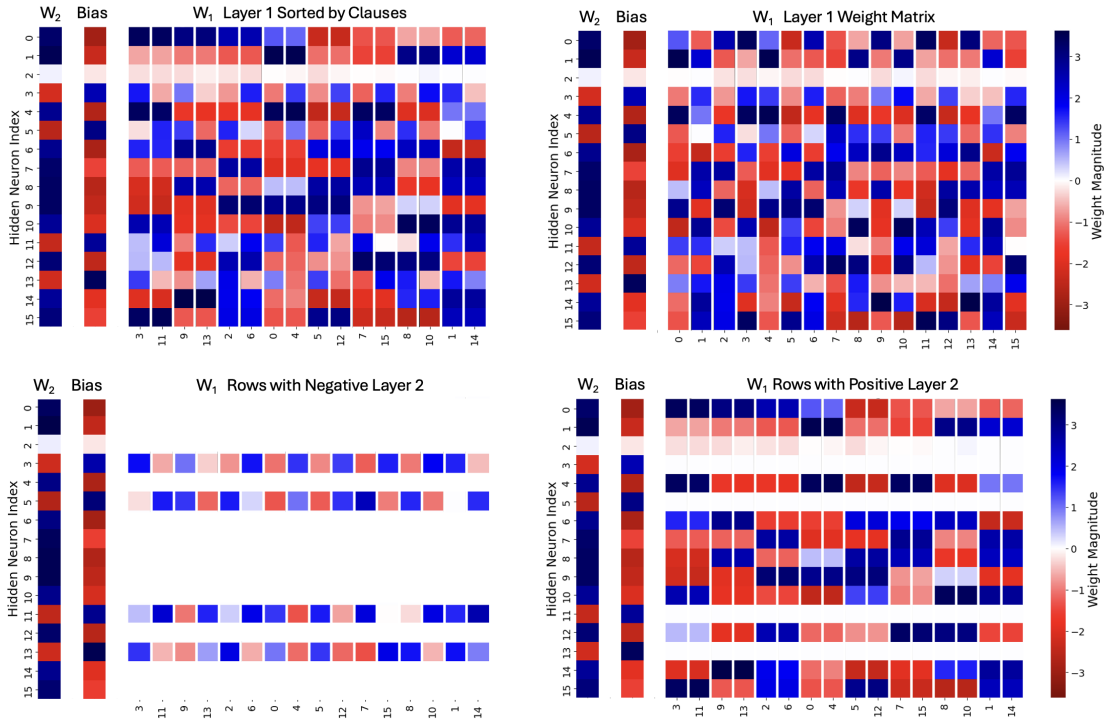

Figure 2: All weights and biases for a neural network trained on the Boolean formula $(x_3 \wedge x_{11}) \vee (x_9 \wedge x_{13}) \vee (x_2 \wedge x_6) \vee (x_0 \wedge x_4) \vee (x_5 \wedge x_{12}) \vee (x_7 \wedge x_{15}) \vee (x_8 \wedge x_{10}) \vee (x_1 \wedge x_{14})$

We here demonstrate that this does actually happen: a similar algorithm emerges naturally from training using gradient descent. Let us turn to a specific example of this. We constructed a two layer MLP, with 16 inputs, 16 neurons at the first layer, and a single neuron at the second layer, with a Layer 1 bias but no Layer 2 bias. We then generated a Boolean formula as the hidden function for the network to learn; this was chosen randomly from the set of pairwise ANDs of 16 Boolean variables with the variables paired up so each appears in exactly one AND. We here examine the result (which was typical of all such formulas) of the formula

$$f(x_0, \ldots, x_{15}) = (x_3 \wedge x_{11}) \vee (x_9 \wedge x_{13}) \vee (x_2 \wedge x_6) \vee (x_0 \wedge x_4) \vee (x_5 \wedge x_{12}) \vee (x_7 \wedge x_{15}) \vee (x_8 \wedge x_{10}) \vee (x_1 \wedge x_{14}).$$

We trained the network on this formula, with random inputs, each chosen with equal probability of being a True or False instance of the formula. There were 30,000 random inputs chosen. Each input had between two and six True variables (with equal probability). The training process utilized the BCE loss function and the Adam optimizer with a learning rate of 0.01. Training was conducted over 2000 iterations over the training data (epochs).

In the upper right hand corner of Figure 2, we present the full set of learned parameters of one representative training run of the neural network: Layer 1 weights (W_1), Layer 1 bias, and Layer 2 weights (W_2). W_2 is placed upright for convenience. The colors correspond to the sign of the weights, the bluer they are the more positive, the redder the more negative. Initially, let us focus on

W_1. The weight matrix in the figure is in its raw form without alteration. Its columns correspond to input features, the input values of the variables x_i, and its rows correspond to the neurons in this first hidden layer. The matrix W_1 does not seem to reveal much, other than the fact that neuron 2 does not really participate in the computation. In the upper left corner, we sorted the columns of W_1 by the features (variables) so that the pairs that are joined by an AND are next to each other. So for example, columns 3 and 11 are next to each other because $(x_3 \wedge x_{11})$ is a clause in the Boolean expression the network is trained on. This image starts to reveal that there is something interesting going on with these pairs: in many positions, but not all, adjacent columns have the same values.

In the lower right hand corner, features in W_1 are sorted the same way, but now we only consider the rows (neurons) of the weight matrix W_2 that correspond to positive Layer 2 weights. We will justify this filtering below. This image is striking in that in each and every row, the columns corresponding to an AND feature pair seem to be virtually identical. Moreover, if we look at the positive, blue colored weights, we see an emerging pattern. Looking across all pairs of input features (variables), every pair that appears in the same AND output feature is in fact brought together into a feature channel code. That set of neurons allows the AND feature's computation to propagate to the next layer.

For example, in the underlying computation, input features 3 and 11 participate in the computation of the feature $(x_3 \wedge x_{11})$. Thus, they are mapped to the same feature channel code, which consists of positive weights on the set of neurons $\{0, 4, 6, 10, 12, 15\}$. Feature $(x_9 \wedge x_{13})$ is represented by the code $\{0, 6, 8, 14\}$, and feature $(x_5 \wedge x_{12})$ by the code $\{6, 8, 9, 10, 15\}$.

However, the neural network in Figure 2 does not compute $x_i \wedge x_j$ using exact Boolean logic as in our theoretical example, but rather using soft Boolean logic via $\text{ReLU}(x_i + x_j - b)$ for some b (regular Boolean variables would have $b = 1$). We see in the lower right hand side that for every single row with a positive Layer 2 weight, the bias is significantly negative, representing the $-b$. Thus, like in the pure Boolean algorithm we showed, if two input features in the same AND, say $(x_3 \wedge x_{11})$, are both set to True then their sum (calculated by the dot product of each neuron by the input) will make it through the negative bias and the ReLU as a positive value for all the rows of the code. Consequently, the result from the network will be a "True" after the sigmoid.

Note that this network is polysemantic. For example, neuron 4 reacts positively to four different AND features: $(x_3 \wedge x_{11})$, $(x_0 \wedge x_4)$, $(x_7 \wedge x_{15})$, and $(x_1 \wedge x_{14})$. Thus, the result of neuron 4 will be similar when the input consists of 1s for only variables x_3 and x_{11} (a True instance) to what it is when the input consists of 1s for only variables x_3 and x_4 (a False instance). This is where feature channel coding plays a crucial role. The outcome of the computation for a given AND output feature is dependent on all of the feature's coding rows, and these codes are learned in such a way that even if they overlap in some rows, they don't overlap in too many of them. For example, the codes for inputs x_3 and x_4 overlap in neurons 0 and 4, and so when both are 1s they will produce some positive signal due to these two rows, but not nearly as much as when x_3 and x_{11} are both 1s, since the codes for those two inputs overlap in 6 rows.

However, since some positive signal does get through even with the False instance, we see that the negative rows do play a role in the computation. We provide quantitative evidence of this below for various different problems; the degree of importance of the negative roles seems to depend on the specifics of the problem being computed. Here, we see that a strong negative signal is provided by neuron 3 in the case where the input is 1s only on variables x_3 and x_4. Specifically, neuron 3 passes a positive signal to Layer 2 because row 3 is positive in both column 3 and column 4, and that positive value, when multiplied by the negative value in W_2, results in a strong negative